

MagicCode Super-Resolution Software Migration Guide

Document Version: v2.1.0. Release Date: September 4, 2026.

1. Purpose

This guide helps customers migrate from earlier versions (v1.x, v2.0) to MagicCode Super-Resolution Software v2.1.0.

2. Main changes

Area	Legacy (v1.x / v2.0)	v2.1.0	Action
Version	v1.2.0 / v2.0.0	v2.1.0	Rebuild against new headers/libs.
Deliverables	Separate core & enable archives	Single static library per platform	Link only libmagic_sr.a / libmagic_sr.lib.
Headers	mc_interface.h and mc_enable.h	Single header interface/mc_interface.h	Remove mc_enable.h.
API	MC_Init, MC_Process, MC_Uninit, MC_Control	MC_Enable, MC_Disable, MC_GetVersion	Use MC_Enable for all execution/config; MC_Disable for cleanup.
Control	control_param_t and cmd_params_e	Pass updated input_param_t to MC_Enable	Remove control_param_t.
Modes	Spatial / Temporal modes	SPATIAL_* and TEMPORAL_* (0..3)	Numeric values 0..3 preserved.

Temporal Super-Resolution

v2.1.0 provides temporal super-resolution through the unified public C ABI: libmagic_sr.a (Windows: libmagic_sr.lib) + interface/mc_interface.h + MC_Enable / MC_Disable. A single function, MC_Enable, handles implicit handle initialization, dynamic reconfiguration, temporal execution, and execution status query.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGL currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced (Windows Vulkan / Direct3D 11 / desktop GL; Apple Metal) can run edge-resolve; if enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- magic_frame_t.image_in / image_out: caller-owned color and output GPU resources.
- magic_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at initial MC_Enable: Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL (system default); GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on input_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

3. Native code migration

1. Include interface/mc_interface.h.
2. Link libmagic_sr.a (or libmagic_sr.lib). Drop libmagic_sr_enable.a / libmagic_enable_sr.lib.
3. Initialize and process using MC_Enable(&handle, &frame, ¶m, &status).
4. Release resources using MC_Disable(handle).
5. Verify MC_GetVersion() returns "v2.1.0".